

공시 목록 manifest DB 적재

항목	내용
작업 단계	02
작업 영역	데이터 적재(Data Ingestion)
원본 데이터	manifest.jsonl
대상 테이블	PostgreSQL disclosures
대상 데이터셋	contest-2026q1-v1
작성일	2026-08-03
코드 상태	구현 완료, compileJava 성공
DB 상태	실제 Flyway V2 적용 및 공시 4,204건 적재 대기

1. 작업 목적

대회에서 제공한 manifest.jsonl의 공시 목록 4,204건을 검증하고, 기존 companies 테이블의 기업과 연결하여 PostgreSQL disclosures 테이블에 먹등하게 적재한다.

이번 단계에서 저장하는 대상은 공시 원문이 아니라 다음과 같은 메타데이터다.

- 공시 접수번호와 데이터셋 문서 ID
- 공시 기업
- 공시 대분류와 원본 세부 유형
- 보고서명, 접수일과 제출인
- 정정공시 여부
- 정기공시 기준연도와 기준월
- 원문 폴더 상대경로, 파일 형식과 예상 파일 수
- 데이터 출처와 데이터셋 버전

XML·HTML·PDF 원문 내용, 원문 파일 해시, 섹션·문단·표, 검색 청크와 구조화 사실은 이번 단계에서 저장하지 않는다.

2. 원본 데이터 구조

manifest.jsonl은 JSON 배열이 아니라 한 줄에 공시 한 건이 들어 있는 JSON Lines 형식이다.

1번째 줄 → 공시 1건
2번째 줄 → 공시 1건
...
4204번째 줄 → 공시 1건

대표 입력은 다음과 같다.

```
{
  "doc_id": "exchange_20240424800596",
  "corp_code": "00164779",
  "corp_name": "SK하이닉스",
  "listed_name": "SK하이닉스",
  "stock_code": "000660",
  "industry": "IT",
  "sector": "반도체·전자부품",
  "doc_group": "exchange",
  "doc_subtype": "신규시설투자등",
  "report_nm": "신규시설투자등",
  "is_correction": false,
  "rcept_no": "20240424800596",
  "rcept_dt": "20240424",
  "flr_nm": "SK하이닉스",
  "base_year": null,
  "base_month": null,
  "file_path": "raw/exchange/SK하이닉스/20240424800596",
  "file_format": "xml",
```

```
"n_files": 1
}
```

확정된 데이터 집계는 다음과 같다.

항목	건수
전체 공시	4,204
정기공시 periodic	1,054
주요사항보고서 major	598
거래소공시 exchange	1,469
지분공시 holding	1,083
정정공시	1,004
XML	4,201
PDF+HTML	3
연결 기업	70

3. 전체 처리 흐름



호출 순서는 다음과 같다.

```
ContestDisclosureImporterRunner.run()
→ ContestDisclosureImporter.importDisclosures()
→ DisclosureManifestReader.read(manifestPath)
→ 각 JSON 줄을 DisclosureManifestRow로 변환·검증
→ doc_id와 rcept_no 중복 검증
→ 데이터셋 전체·그룹별 건수 검증
→ Company 70개 조회 및 연결 검증
→ 기존 Disclosure와 식별자 비교
→ 신규·변경·동일 데이터 판별
→ 신규·변경 데이터만 저장
→ ImportResult 반환 및 로그 출력
```

4. Flyway 테이블

마이그레이션 파일은 다음과 같다.

```
backend/src/main/resources/db/migration/V2__create_disclosures.sql
```

V2는 다음을 생성한다.

- **disclosures** 테이블
- **companies.id**를 참조하는 **company_id** 외래키
- **source_doc_id**와 **receipt_no** 고유 제약
- 공시 그룹, 카테고리, 접수번호, 기준기간, 파일 형식과 상대경로 검사 제약
- 기업·카테고리·세부 유형·정정공시·데이터셋 버전 조회 인덱스

기업과 공시는 다음 일대다 관계다.

```
Company 1 — N Disclosure
```

하나의 공시는 반드시 하나의 기업에 연결되며, 공시가 존재하는 기업은 **ON DELETE RESTRICT** 때문에 삭제할 수 없다.

5. manifest와 DB 컬럼 매핑

manifest 필드	disclosures 컬럼	처리 방식
doc_id	source_doc_id	데이터셋 문서 ID, UNIQUE
corp_code	company_id	companies.corp_code로 조회 후 FK 저장
doc_group	source_group	원본 소문자 그룹 보존
doc_group	category	서비스 대분류로 변환
doc_subtype	raw_subtype	원본 세부 유형, null 허용
report_nm	report_name	원본 보고서명
is_correction	correction	정정공시 여부
rcept_no	receipt_no	DART 접수번호, UNIQUE
rcept_dt	receipt_date	yyyyMMdd를 LocalDate로 변환
flr_nm	submitter	제출인
base_year	base_year	정기공시 기준연도
base_month	base_month	정기공시 기준월
file_path	manifest_path	데이터셋 루트 기준 상대경로
file_format	file_format	xml 또는 pdf+html
n_files	expected_file_count	폴더의 예상 파일 수

다음 값은 애플리케이션에서 추가한다.

컬럼	입력값
id	UUID 자동 생성
source_provider	CONTEST
source_dataset_version	기본값 contest-2026q1-v1
created_at	JPA Auditing 또는 DB 기본값
updated_at	JPA Auditing 또는 DB 기본값

corp_name, listed_name, stock_code, industry, sector는 Company에 이미 저장되어 있으므로 disclosures에 중복 저장하지 않는다. 대신 적재 과정에서 Company 값과 일치하는지 검증한다.

6. 도메인 모델

DisclosureCategory

서비스 API에서 사용하는 정규화된 공시 대분류다.

```

PERIODIC
MATERIAL
EXCHANGE
OWNERSHIP

```

DisclosureSourceGroup

manifest의 원본 그룹과 서비스 카테고리를 연결한다.

원본	서비스 카테고리
periodic	PERIODIC
major	MATERIAL
exchange	EXCHANGE
holding	OWNERSHIP

DisclosureFileFormat

```
XML      → DB의 xml
PDF_HTML → DB의 pdf+html
```

두 Enum은 JPA `AttributeConverter`를 사용해 Java 값과 DB 원본 문자열을 변환한다.

Disclosure

`BaseTimeEntity`를 상속하고 `Company`와 지연 로딩 `ManyToOne` 관계를 가진다. 생성 메서드는 원본 그룹에서 카테고리를 자동 결정하며, 식별자-기준기간-상대경로-파일 수를 검증한다.

변경 가능한 메타데이터는 `updateMetadata()`로만 갱신한다. 다음 식별 정보는 기존 공시에서 변경하지 않는다.

- `sourceDocId`
- `receiptNo`
- `company`
- `sourceGroup`
- `sourceProvider`

7. DisclosureManifestRow

manifest 한 줄을 나타내는 Java record다.

주요 역할은 다음과 같다.

- Jackson `@JsonProperty`로 JSON 필드와 Java 필드 연결
- `receipt_dt`를 `LocalDate`로 변환
- 필수값과 문자열 길이 검증
- 기업코드-종목코드-접수번호 형식 검증
- `doc_id = doc_group + '_' + receipt_no` 검증
- 정기공시 기준연도-월 검증
- 안전한 상대경로와 파일 형식 검증
- Company 값 일치 검증
- 신규 Disclosure 생성과 기존 Disclosure 갱신

검증은 별도 Bean Validation 구조를 추가하지 않고 record의 compact constructor와 직접 검증 메서드로 처리했다.

8. DisclosureManifestReader

`manifest.jsonl`을 UTF-8로 한 줄씩 읽는다. 프로젝트가 Spring Boot 4.1과 Jackson 3을 사용하므로 다음 `ObjectMapper`를 주입한다.

```
tools.jackson.databind.ObjectMapper
```

처리 내용은 다음과 같다.

1. 경로가 존재하고 읽을 수 있는 일반 파일인지 확인
2. 첫 번째 줄의 UTF-8 BOM 제거
3. 빈 줄 거부
4. 각 줄을 `DisclosureManifestRow`로 변환
5. Row 검증 오류에 실제 manifest 줄 번호 추가
6. 파일 내부의 `doc_id`, `receipt_no` 중복 검사
7. 변경 불가능한 `List` 반환

Reader는 파일 구조와 파일 내부 중복만 책임진다. 전체 데이터셋 건수와 DB 연결은 `Importer`가 검증한다.

9. DisclosureRepository

`JpaRepository<Disclosure, UUID>`를 상속하며 다음 조회를 제공한다.

- `findBySourceDocId()`
- `findByReceiptNo()`
- `findAllWithCompany()`

Importer는 `JOIN FETCH` 기반 `findAllWithCompany()`를 사용해 기존 공시와 Company를 한 번에 조회한다. 공시마다 Company 추가 SQL을 실행하는 N+1 문제를 방지한다.

10. ContestDisclosureImporter

Importer는 다음 고정 데이터 품질 기준을 먼저 확인한다.

- 전체 4,204건
- 그룹별 1,054 / 598 / 1,469 / 1,083건
- 정정공시 1,004건
- XML 4,201건
- PDF+HTML 3건
- 고유 기업 70개
- DB Company 70개

기존 DB 데이터는 다음 Map으로 구성한다.

```
corpCode    → Company
sourceDocId → Disclosure
receiptNo   → Disclosure
```

각 manifest Row는 다음 기준으로 처리한다.

신규

`sourceDocId`와 `receiptNo`가 모두 DB에 없으면 Disclosure를 생성한다.

미변경

기존 Disclosure와 모든 메타데이터가 같으면 저장하지 않고 `unchangedCount`만 증가시킨다.

변경

식별 정보는 같고 변경 가능한 메타데이터가 다르면 기존 Entity를 갱신한다.

충돌

다음 경우 전체 적재를 실패시킨다.

- `sourceDocId`와 `receiptNo` 중 하나만 DB에 존재
- 두 식별자가 서로 다른 DB 공시를 가리킴
- 기존 공시가 다른 Company에 연결됨
- manifest 기업 정보가 Company와 불일치

전체 작업에 `@Transactional`을 적용했기 때문에 한 건이라도 실패하면 이번 적재 전체가 롤백된다.

11. ApplicationRunner와 실행 순서

공시 적재 Runner는 다음 설정이 `true`일 때만 등록된다.

```
FOLIOLENS_IMPORT_DISCLOSURES_ON_STARTUP=true
```

기업 적재가 먼저 실행되어야 하므로 Runner 순서는 다음과 같다.

```
@Order(1) ContestCompanyImportRunner
@Order(2) ContestDisclosureImportRunner
```

기업 데이터가 이미 70건 적재되어 있으면 기업 적재 옵션은 꺼도 된다.

12. Docker 환경설정

호스트 데이터셋 디렉터리는 애플리케이션 컨테이너의 `/data`에 읽기 전용으로 연결한다.

```
호스트 ../foliolens-data
→ 컨테이너 /data
→ /data/manifest.jsonl
```

관련 환경변수는 다음과 같다.

```
FOLIOLENS_DATASET_PATH=../foliolens-data
FOLIOLENS_IMPORT_COMPANIES_ON_STARTUP=false
FOLIOLENS_IMPORT_DISCLOSURES_ON_STARTUP=true
```

13. 실행 방법

사전 조건

- Docker Desktop 실행
- `companies` 테이블에 기업 70건 존재
- `../foliolens-data/manifest.jsonl` 존재
- 실제 `.env`에서 공시 적재 옵션 활성화

새 코드를 Docker 이미지에 포함해 실행한다.

```
docker compose up --build
```

백그라운드 실행은 다음과 같다.

```
docker compose up -d --build
docker compose logs -f app
```

최초 적재 성공 시 예상 로그는 다음과 같다.

```
Contest disclosure manifest import started.
Contest disclosure manifest import completed. input=4204, created=4204, updated=0, unchanged=0, total=4204
```

같은 데이터 재실행 시 예상 결과는 다음과 같다.

```
input=4204, created=0, updated=0, unchanged=4204, total=4204
```

최초 적재가 완료되면 매번 실행하지 않도록 옵션을 다시 끈다.

```
FOLIOLENS_IMPORT_DISCLOSURES_ON_STARTUP=false
```

14. DB 검증 SQL

Flyway V2 적용 여부를 확인한다.

```
SELECT version, description, success
FROM flyway_schema_history
ORDER BY installed_rank;
```

전체 공시 수를 확인한다.

```
SELECT COUNT(*)
```

```
FROM disclosures;
```

예상 결과는 4,204다.

서비스 카테고리별 건수를 확인한다.

```
SELECT category, COUNT(*)
FROM disclosures
GROUP BY category
ORDER BY category;
```

category	예상 건수
PERIODIC	1,054
MATERIAL	598
EXCHANGE	1,469
OWNERSHIP	1,083

추가 검증 SQL은 다음과 같다.

```
-- 연결 기업 수: 70
SELECT COUNT(DISTINCT company_id)
FROM disclosures;

-- 정정공시 수: 1,004
SELECT COUNT(*)
FROM disclosures
WHERE correction = TRUE;

-- 파일 형식별 건수: xml 4,201 / pdf+html 3
SELECT file_format, COUNT(*)
FROM disclosures
GROUP BY file_format
ORDER BY file_format;

-- 중복 결과가 없어야 함
SELECT source_doc_id, COUNT(*)
FROM disclosures
GROUP BY source_doc_id
HAVING COUNT(*) > 1;

SELECT receipt_no, COUNT(*)
FROM disclosures
GROUP BY receipt_no
HAVING COUNT(*) > 1;
```

15. 현재 검증 상태

2026-08-03 기준 상태는 다음과 같다.

- Disclosure 관련 코드 구현 완료
- `application.yml`, `compose.yml`, `.env.example` 실행 옵션 연결 완료
- 로컬 `gradlew compileJava --no-daemon` 성공
- 이전 Docker 빌드는 미완성 `QuestionPlan` 타입 때문에 컴파일 단계에서 실패
- 질문 도메인 수정 후 로컬 Java 컴파일 성공 확인
- 현재 실행 중인 Docker 컨테이너 없음
- 따라서 Flyway V2 실제 적용과 공시 4,204건 DB 적재 성공은 아직 확인하지 않음

구현 완료와 실제 데이터 적재 완료는 별개의 상태다. 다음 Docker 실행에서 Runner 완료 로그와 위 SQL 결과를 모두 확인해야 이 단계를 최종 완료로 판단한다.

16. 관련 파일

파일	역할
<code>docs/DATA_CATALOG.md</code>	원본 데이터 형식과 품질 기준
<code>backend/src/main/resources/db/migration/V2__create_disclosures.sql</code>	공시 메타데이터 테이블 생성
<code>backend/src/main/java/com/foiliolens/backend/disclosure/domain/Disclosure.java</code>	공시 Entity
<code>backend/src/main/java/com/foiliolens/backend/disclosure/domain/DisclosureCategory.java</code>	서비스용 대분류
<code>backend/src/main/java/com/foiliolens/backend/disclosure/domain/DisclosureSourceGroup.java</code>	원본 그룹과 카테고리 매핑
<code>backend/src/main/java/com/foiliolens/backend/disclosure/domain/DisclosureFileFormat.java</code>	원문 형식 Enum
<code>backend/src/main/java/com/foiliolens/backend/disclosure/domain/converter/*</code>	Enum과 DB 문자열 변환
<code>backend/src/main/java/com/foiliolens/backend/disclosure/repository/DisclosureRepository.java</code>	공시 DB 접근
<code>backend/src/main/java/com/foiliolens/backend/disclosure/infrastructure/dataset/DisclosureManifestRow.java</code>	manifest 한 줄 표현·검증·변환
<code>backend/src/main/java/com/foiliolens/backend/disclosure/infrastructure/dataset/DisclosureManifestReader.java</code>	JSONL 읽기·줄 번호 중복 검사
<code>backend/src/main/java/com/foiliolens/backend/disclosure/infrastructure/dataset/ContestDisclosureImporter.java</code>	전체 품질 검증·기업 연결·역등 저장
<code>backend/src/main/java/com/foiliolens/backend/disclosure/infrastructure/dataset/ContestDisclosureImportRunner.java</code>	서버 시작 시 조건부 재
<code>backend/src/main/resources/application.yml</code>	데이터셋 경로·버전·실행 옵션
<code>compose.yml</code>	DB·앱 실행과 데이터셋 볼륨 연결
<code>.env.example</code>	로컬 환경 변수 예시

17. 다음 단계

1. Docker Desktop 실행
2. 실제 `.env`에서 공시 적재 옵션 활성화
3. `docker compose up --build`
4. Runner의 4,204건 완료 로그 확인
5. SQL로 Flyway V2-전체 건수·그룹별 건수·중복 여부 검증
6. 자동 적재 옵션 비활성화
7. 이후 실제 원문 파일을 관리할 `disclosure_documents` 설계 및 적재 진행